



Codex 双开小白指南

一个环境变量，把 Codex 变成两个互不认识的人

CODEX_HOME 多实例隔离 · Windows 实测版

来源：X 文章《Codex双开配置教程》（Hardy Chen @Chchenhao0129，2026-09-18，97 赞 / 218 书签），经诺鸭船长3 @noahduck283 引用传播 · 本指南全部命令在 Windows 11 + codex-cli 0.160.0 真机跑通 · 2026-10-03

💡 **一句话读懂全篇**：Codex 所有"私人物品"——登录凭证、配置、会话历史、记忆——全都放在一个叫 `CODEX_HOME` 的家目录里（默认 `~/.codex`）。启动前把这个环境变量改指到另一个目录，就得到一个全新身份的 Codex，两个实例互不知道对方存在。

? 十三个疑问，从浅到深

1 "双开"到底是什么？为什么要双开？

双开 = 在同一台电脑上同时运行两个完全独立的 Codex 实例。原教程的动机很直接：Plus 订阅额度不够用、Pro 又太贵。但双开的真正价值不止于此：

- **身份分家**：工作账号干公司活，个人账号干私活，凭证互不干扰；
- **配置分家**：一个实例配保守小模型+严格沙箱，另一个放开手脚跑实验，**改坏了互不牵连**；
- **记忆分家**：写代码的 Codex 和写文章的 Codex，会话历史/记忆各存各的。

实例① 打工型

`~/.codex` · 工作账号 · 严格沙箱

同一台电脑

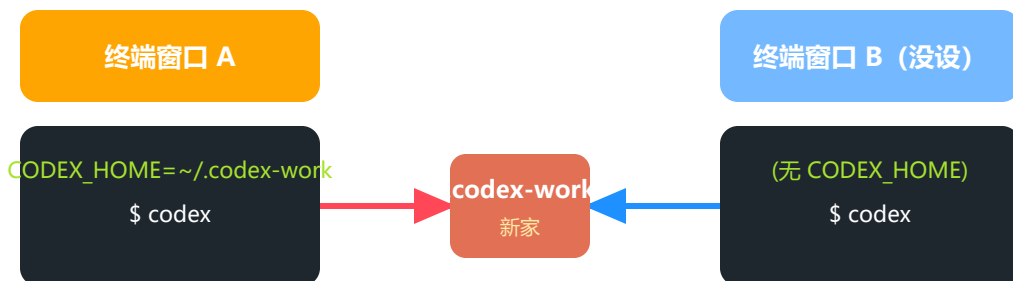
实例② 实验型

`~/.codex-work` · 个人账号 · 放开跑

2 凭什么改个环境变量就能"分家"?

环境变量是**每个终端进程自己的口袋便签**：你在 A 窗口写的变量，B 窗口看不见。Codex 每次启动的第一件事，就是掏出便签看一句话——**"我家在哪?"**

- 没写 `CODEX_HOME` → 用默认家 `~/.codex` (Windows 即 `C:\用户名\.codex`) ;
- 写了 → 用你指的目录当家，**凭证、配置、历史全部从这个新目录读**。



同一个 `codex.exe`，凭窗口里的环境变量各回各家

讲完比喻落回术语：`CODEX_HOME` 是 **Codex 官方支持的配置项**（不是野路子 hack），官方文档明确它决定配置、认证、会话数据的存放位置。

3 CODEX_HOME 这个"家"里到底住着什么?

本机实测 `~/.codex` 的清单（节选）——这就是"分家"到底分走了什么：

文件/目录	是什么	换家后
<code>auth.json</code>	登录凭证（ChatGPT 账号身份）	另一个身份
<code>config.toml</code>	模型、沙箱、审批等全部配置	另一套配置
<code>history.jsonl</code>	输入历史	各记各的
<code>archived_sessions/</code> 等	会话存档（resume 的原材料）	互不可见
<code>memories*.sqlite</code>	长期记忆库	互不串味
<code>logs_2.sqlite</code>	运行日志	各自记账

家 = 身份 + 配置 + 记忆的打包整体，换家 = 换一个"人"。

4 为什么我一设置就报错罢工？（必踩的坑）

🐞 **本机实测坑：**直接设 `CODEX_HOME` 指向一个不存在的目录，Codex 不会自动创建它，而是拒绝启动：

```
WARNING: proceeding, even though we could not create PATH aliases:
CODEX_HOME points to "C:\Users\Windows\.codex-work", but that path does not exist
Error loading configuration: CODEX_HOME points to ... but that path does not exist
```

对策一行：**启动前先建目录**。好的启动器脚本都会带这句兜底：

```
# PowerShell
New-Item -ItemType Directory -Force "$env:USERPROFILE\.codex-work"
$env:CODEX_HOME = "$env:USERPROFILE\.codex-work"; codex
```

建好后第一次运行，新家从空目录开始生长——这本身就是隔离生效的铁证。

5 二号实例怎么登录？一号会掉线吗？

不会掉线。凭证存在各自的 `auth.json` 里，互不相干。二号第一次用时在自己的窗口里正常 `codex login`（浏览器登录另一个账号，或配 API key）。本机实测两家体检报告对比：

<pre># 老家（默认） \$ codex login status Logged in using ChatGPT \$ codex doctor（配置段） config.toml C:\...\codex\config.toml auth file C:\...\codex\auth.json parse ok ✓</pre>	<pre># 新家（CODEX_HOME=.codex-work） \$ codex login status Not logged in \$ codex doctor（配置段） config.toml C:\...\codex-work\config.toml auth file C:\...\codex-work\auth.json parse ok ✓</pre>
---	---

doctor 直接报出两家的 config 路径和 auth 路径——隔离是文件系统级的，不是界面障眼法。

6 config.toml 里不是有 profiles 吗？跟双开什么关系？

profiles 是同一个家里的"不同穿搭方案"：在 config.toml 里写多个 `[profiles.名字]`，用 `codex --profile 实验装` 切换模型/沙箱等设置。但它不换 `auth.json`、不换历史、不换记忆。

	profiles（换穿搭）	CODEX_HOME 双开（换人）
模型/沙箱等配置	可多套切换	可多套
登录账号	共享同一个	各自独立
会话历史 / 记忆	共享	完全隔离
同时在线两账号	做不到	开两个窗口即可

口诀：只换配置用 profiles，要换身份必须双开。

7 会话、记忆、MCP 服务器配置也一起隔离吗？

是，全在 CODEX_HOME 这一个屋檐下，跟着家走。实测：新家 `.codex-work` 初始只有一个 `tmp/`，老家的 `memories`、`history`、MCP 配置一概不在。另外还有一个单次粒度的轻量开关：`codex -c model="xxx"` 只在这一次调用里覆盖配置，落盘不生效。三层覆盖关系：



隔离粒度从左到右递增：单次 → 配置 → 身份+配置+记忆

8 原教程是 macOS 的，跟我 Windows 用户有什么关系？

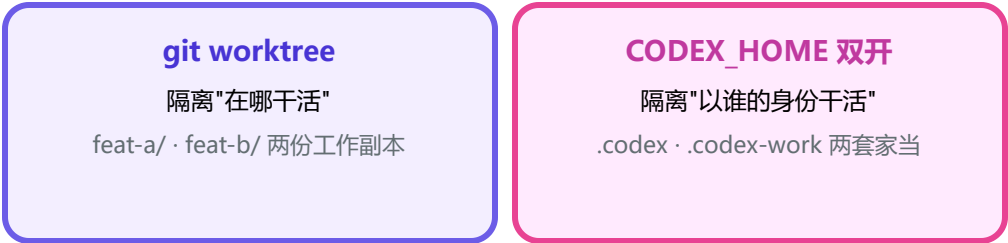
原文针对 Mac 桌面版 Codex App： `open -n -a Codex` 强制开第二个应用实例 + `CODEX_ELECTRON_USER_DATA_PATH` 分数据目录 + AppleScript 做独立启动器。两条路线对比：

	macOS 桌面版（原教程）	CLI 双开（本指南，跨平台）
关键变量	<code>CODEX_HOME</code> + <code>CODEX_ELECTRON_USER_DATA_PATH</code>	<code>CODEX_HOME</code> 一个就够
启动方式	<code>open -n -a Codex</code> + AppleScript	脚本设变量后直接 <code>codex</code>
Windows 可用性	桌面 App 双开在 Windows 上可靠性存疑（社区反馈新版本下旧法失效）	实测稳：本指南全部产出

CLI 是双开的“公共底层”——两套生态方法共享同一个原理：换 `CODEX_HOME` 即换人。

9 双开和 git worktree 并行隔离是什么关系？

完全正交（互不重叠、可叠加）的两层隔离：worktree 隔离的是代码工作目录（防止两个任务改同一份文件），CODEX_HOME 隔离的是身份与配置（防止两个实例共用账号/记忆/设置）。



组合拳：实例② 在 worktree feat-b/ 里用自己的账号干活

(知识库里已有 [[GitWorktree并行隔离]] 一格，本篇补上另一格。)

10 有更轻的做法吗？什么时候轮不到双开？

按"改动量"从小到大选工具，别用大炮打蚊子：

需求	用哪个	成本
就想这一次换个模型	<code>codex -c model="..."</code>	零
常备两套模型/沙箱方案，同一账号	<code>config.toml</code> profiles	低
要两个账号同时在线 / 记忆历史分开	CODEX_HOME 双开	中（多一个登录）
多任务并行改同一仓库的代码	git worktree（可叠加双开）	中

11 升级/回滚时两个实例会打架吗？

程序是共享的，家当是分开的。`npm i -g @openai/codex` 装的 `codex.exe` 全机只有一份，两个实例运行的是同一个版本；双开分走的只是数据目录。所以：

- 升级一次，两实例一起升级；
- 原教程讲的"更新回滚流程"，CLI 上的等价物是装指定版本：`npm i -g @openai/codex@0.159.0`；
- 某版本翻车时，两实例的配置/凭证不受程序回滚影响（它们在各自家里，不随 exe 走）。

12 双开能多搞一份订阅额度吗？（诚实版答案）

不能。额度跟着 ChatGPT 账号走，不跟实例走。双开的两个实例登录同一个账号，烧的是同一份额度；登录两个账号，各烧各的、也各付各的钱。原教程说的"plus 不够用"场景，真实含义是**把两个账号的额度用于不同工作流，避免互相挤兑**，而不是凭空变出额度。双开的正当收益是：**隔离（互不污染）与并行（同时在线两身份）**，不是套利。结合知识库 [[规划执行分账]]：省额度的正道是"贵推理挪网页版、执行留给 CLI"，与双开正交。

13 这个"环境变量分家"思想还能举一反三到哪？

"换家目录 = 换身份"是 CLI 工具界的通用模式，一大批工具都认同一个道理：

工具	家目录变量	默认位置
OpenAI Codex CLI	CODEX_HOME	~/.codex
Claude Code	CLAUDE_CONFIG_DIR	~/.claude
git (全局配置+凭证)	GIT_CONFIG_GLOBAL / HOME	~/.gitconfig
SSH	-F 指定 config	~/.ssh
Docker	DOCKER_CONFIG	~/.docker

学会一次，以后看到任何工具"多账号/多环境"需求，第一反应就应该是：找它的家目录变量。

主实验：亲手把 Codex 分成两个人（Windows 真机全部跑通）

实验环境：Windows 11 · codex-cli 0.160.0 (npm 全局安装) · 全程不写一行代码，只用 PowerShell。目标：30 分钟后你拥有两个互相不知道对方存在的 Codex。

第 1 步 · 先认清"老家的身份指纹"（基线）

操作（新开 PowerShell）：

```
codex login status
Get-Content "$env:USERPROFILE\.codex\config.toml"
```

目的：记住默认实例的登录状态和配置，等下对比用。实现了什么：确认"实例①"现在长什么样。

✅ 预期结果：Logged in using ChatGPT；config.toml 里能看到 model = "gpt-6.1-sol" 之类的行（本机实测）。

第 2 步 • 开新家（并踩掉那个必踩的坑）

操作：

```
$env:CODEX_HOME = "$env:USERPROFILE\.codex-work"
codex login status      # 先故意不建目录，看它罢工
Error loading configuration: ... but that path does not exist

New-Item -ItemType Directory -Force "$env:USERPROFILE\.codex-work"
codex login status
```

目的：验证凭据隔离。**实现了什么：**同一个 codex.exe 在新家变成一个未登录的新身份。

✅ 预期结果：建目录前报 **"path does not exist"**（坑！）；建目录后输出 **Not logged in**。再看看新家：里面只有一个 tmp\ —— 干净得像刚出生。

第 3 步 • 给新家一份自己的配置，并用 doctor 出示"户口证明"

操作：

```
Set-Content "$env:USERPROFILE\.codex-work\config.toml" @"
model = "gpt-6.1-sol"
model_reasoning_effort = "high"
"@
codex doctor | Select-String "config.toml|auth file"
```

目的：证明配置文件也是各读各的。**实现了什么：**新家 reasoning_effort=high，老家仍是 low，互不影响。

✅ 预期结果：doctor 明确报出 **config.toml C:\...\codex-work\config.toml** 和 **auth file C:\...\codex-work\auth.json**，parse ok。把 **\$env:CODEX_HOME** 删掉（**Remove-Item Env:CODEX_HOME**）再跑 doctor，路径立刻变回老家——切换零成本。

第 4 步 • 做一键启动器，把方法固化下来

操作：把下面存成 `codex-work.ps1` (完整版已附在 exercise\ 目录，另有双击即用的 `codex-work.cmd`)：

```
$env:CODEX_HOME = Join-Path $env:USERPROFILE ".codex-work"
if (-not (Test-Path $env:CODEX_HOME)) { New-Item -ItemType Directory -Force $env:CODEX_HOME | Out-Null }
codex @args
```

目的：以后每次想进"实例②"，双击/一行命令即可，不用记变量名。**实现了什么：**方法从"一次性操作"变成"常备工具"。

✓ 预期结果：`powershell -File codex-work.ps1 doctor` 的输出指向 `.codex-work` (本机实测通过)。到此：实例①开普通终端 `codex`，实例②跑 `codex-work.ps1`，两个窗口同时在线互不打架。

🎁 这套方法给我什么

🏠 工作/私人双账号同机在线，凭证永不串

🔧 野实验专用实例：配置随便改，老家永远安全

📄 写作/编码记忆分家，resume 历史互不污染

🔗 与 git worktree 叠加 = 目录+身份双层并行

📁 通用模式迁移：CLAUDE_CONFIG_DIR / DOCKER_CONFIG 同理

📌 **三句话带走：**① Codex 的家当全在 `CODEX_HOME`，换家即换人；② 家目录必须先建好，否则它罢工；③ 隔离的是身份/配置/记忆，不是额度——省额度请找 [[规划执行分账]] 那套办法。